

CIS761 Class Project- Database Application Project proposal

Matin Marjani

Title:

Expense Management System

Description

The system is a personal finance management application. It allows users to record their financial activity across multiple accounts, classify transactions using categories and tags, and attach receipts as supporting evidence. The system assumes a single currency throughout. The main entity sets are Users, Accounts, Categories, Tags, Transactions, and Receipts. Beyond basic transaction management, the system will support analytical and visualization features such as category-based spending summaries, account balance and asset trends, calendar views, and advanced search and filtering. This project is well suited for a relational database because it includes multiple related entities, clear integrity constraints, and many useful query and reporting scenarios. The final system can be implemented as a simple web or mobile application that demonstrates both database functionality and a practical user interface.

Entities and Attributes

User

A user represents a person who owns and manages financial data. user owns all accounts, categories, and tags that they create, and records all transactions within their accounts.

Attributes:

user_id: primary key

email: unique, required

username: unique, required

password_hash: required

full_name: required

created_at: required

Account

account represents a container owned by a user, like checking account, savings account, cash wallet or credit card. Each account belongs to exactly one user. The account's current balance is not stored, it is computed from the transactions associated with the account.

Attributes:

account_id: primary key

name: required (e.g., "Chase Checking")

type: required; enumerated value: checking, savings, cash, credit_card

initial_balance: required (the balance at the time the account was created)

created_at: required

Uniqueness constraint: (user_id, name) must be unique because a user cant have two accounts with the same name.

Category

category is to classify transactions of type income or expense. Each category belongs to exactly one user and is typed as either an income category or an expense category. Example income categories: Salary, Refunds. Example expense categories: Groceries, Rent, Transportation.

Attributes:

category_id: primary key

name: required (e.g., "Groceries")

type: required; value: income, expense

Uniqueness constraint: (user_id, name) must be unique because a user cant have two categories with the same name.

Tag

tag is a flexible label. a user can label transactions regardless of transaction type or category with any tags. Each tag belongs to exactly one user.

Attributes:

tag_id: primary key

name: required

Uniqueness constraint: (user_id, name) must be unique because a user cant have two tags with the same name.

Transaction

A transaction represents a single financial event recorded by a user within one of their accounts. Every transaction has a type: income, expense, or transfer.

Income transactions represent money entering an account (salary, refunds, ...)

Expense transactions represent money leaving an account (purchases, bills, ...)

Transfer transactions are money moved from one account to another account, where both accounts are owned by the same user.

Attributes:

transaction_id: primary key

type: required; enumerated value: income, expense, transfer

amount: required; must be positive

transaction_date: required (the date the financial event occurred)

entry_date: required (the date the transaction was recorded in the system)

description: optional

Receipt

A receipt is a file, image or scanned document attached to a transaction as evidence. Each receipt belongs to exactly one transaction, and each transaction can have at most one receipt. A receipt is a non-weak entity with its own identity and its own descriptive attributes.

Attributes:

- **receipt_id:** primary key
- **file_name:** required
- **file_type:** required (pdf, jpg, png)
- **storage_location:** required
- **uploaded_at:** required

Relationships

Owns (User — Account)

A user owns many accounts. each account belongs to exactly one user.

Multiplicity: 1 : N (User : Account)

Participation: User partial (a user may have zero accounts), Account total (every account must have an owner).

Defines (User — Category)

A user defines many categories. each category belongs to exactly one user.

Multiplicity: 1 : N (User : Category)

Participation: User partial, Category total.

Creates (User — Tag)

A user creates many tags. each tag belongs to exactly one user.

Multiplicity: 1 : N (User : Tag)

Participation: User partial, Tag total.

Records (Account — Transaction)

Every transaction is associated with exactly one account, its main account. This is the account where the transaction is primarily recorded: for income, the destination account; for expense, the source account; for transfer, the origin of the funds.

Multiplicity: 1 : N (Account : Transaction)

Participation: Account partial, Transaction total.

Counterparty (Account — Transaction)

For transfer transactions only, a transaction also references a second account, the counterparty account, representing the other side of the transfer. This relationship exists exclusively for

transactions of type transfer. Both the main account and the counterparty account must belong to the same user, and they must be different accounts.

Multiplicity: 1 : N (Account : Transaction), but only applicable when Transaction.type = transfer

Participation: Account partial; Transaction partial (only transfers participate).

Classifies (Category — Transaction)

A category may be assigned to a transaction of type income or expense only. Transfer transactions never have a category. When a category is assigned, the category's type must match the transaction's type (an income transaction must use an income category; an expense transaction must use an expense category). Categories on income and expense transactions are optional.

Multiplicity: 1 : N (Category : Transaction)

Participation: Category partial; Transaction partial (transfers never participate, and even income/expense transactions may have no category).

TaggedWith (Transaction — Tag)

A transaction may have multiple tags, and a tag may be attached to many transactions.

Multiplicity: M : N (Transaction : Tag)

Participation: Both partial.

HasReceipt (Transaction — Receipt)

A transaction has at most one receipt, and each receipt belongs to exactly one transaction.

Multiplicity: 1 : 1 (Transaction : Receipt)

Participation: Transaction partial (most transactions have no receipt); Receipt total (every receipt must be attached to a transaction).

Global Constraints and Business Rules

1. A user can only interact with their own data. All categories, tags, accounts, and transactions referenced by a user's transaction must belong to the same user.

Specifically:

A transaction's main account must belong to the transaction's owning user.

A transaction's category (if any) must belong to the same user.

A transaction's tags must all belong to the same user.

For transfers, the counterparty account must belong to the same user as the main account.

2. Transaction-type rules:

If type = income → category, if present, must be of type income; no counterparty account.

If type = expense → category, if present, must be of type expense; no counterparty account.

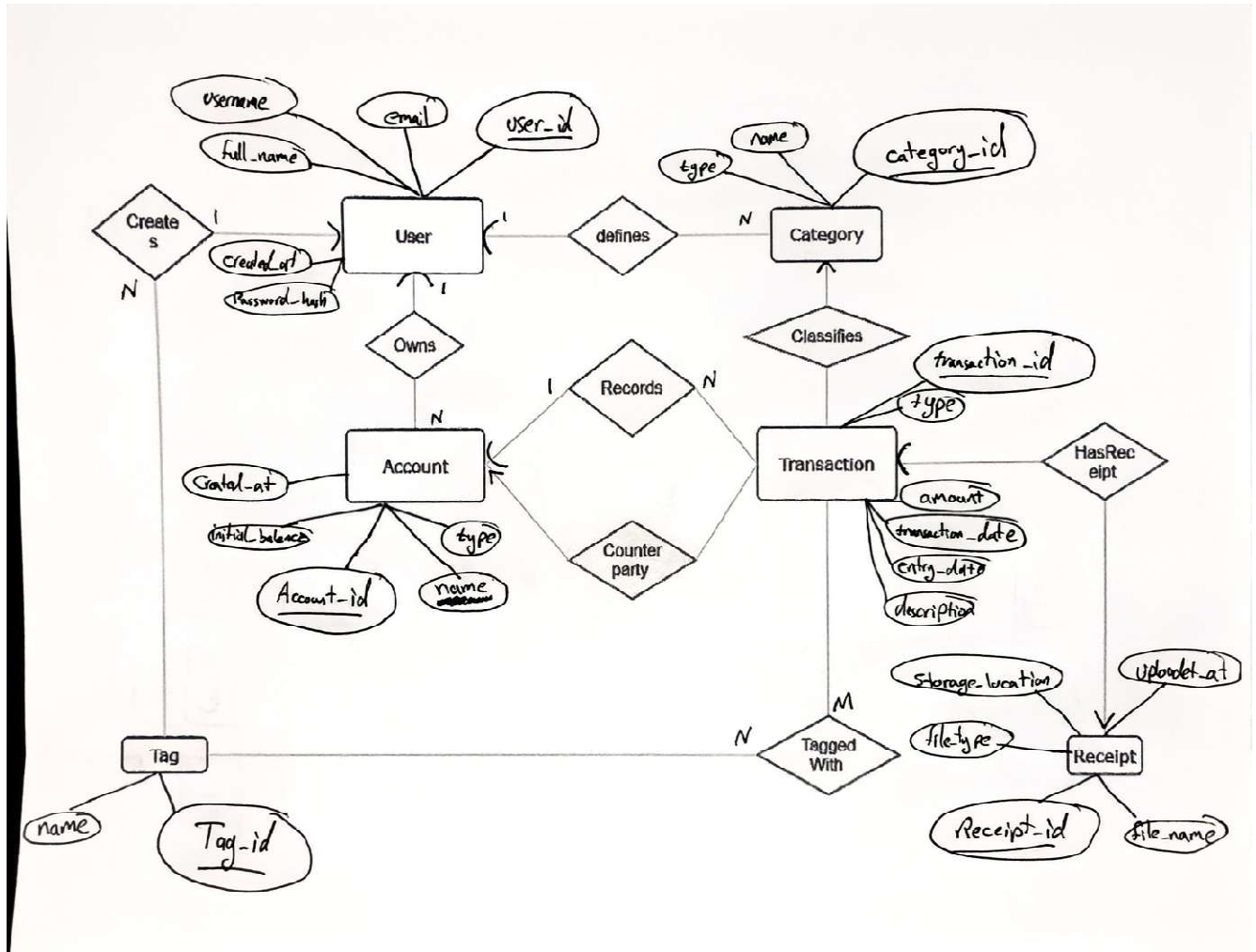
If type = transfer → no category; counterparty account is required and must differ from the main account.

3. Transaction amount must be positive. The direction of money flow is determined by the transaction type, not by the sign of the amount.
4. Account balance is not stored. It is always computed as: $\text{initial_balance} + (\text{sum of incoming transactions}) - (\text{sum of outgoing transactions})$, where transfers contribute to both the source and destination account computations.
5. The system operates in a single currency; no currency attribute is stored on any entity.
6. Uniqueness (beyond primary keys): (user_id, name) is unique within Accounts, Categories, and Tags. Email and username are globally unique within Users.

Data

Since an expense management system is based on everyday financial activity, we can populate the database with realistic sample data even if complete real-world data is not available. The dataset will include multiple users, several financial accounts per user, many transactions across different dates, categories, and tags, and receipt records. If needed, part of the data can be fabricated in a realistic way to reflect common finance behavior. This system has enough depth and variety to support interesting queries, reports, and user interactions in the final demo.

E/R diagram:



Relations:

Users(user_id, email, username, password_hash, full_name, created_at)

PK: user_id

Unique: email, username

Accounts(account_id, user_id, name, type, initial_balance, created_at)

PK: account_id FK: user_id references Users(user_id)

Unique: (user_id, name)

Categories(category_id, user_id, name, type)

PK: category_id

FK: user_id references Users(user_id)

Unique: (user_id, name)

Check: type ∈ {income, expense}

Tags(tag_id, user_id, name)

PK: tag_id

FK: user_id references

Users(user_id)

Unique: (user_id, name)

Transactions(transaction_id, account_id, counterparty_account_id, category_id, type, amount, transaction_date, entry_date, description)

PK: transaction_id

FK: account_id references Accounts(account_id)

FK: counterparty_account_id references Accounts(account_id) — nullable

FK: category_id references Categories(category_id) — nullable

Check: type ∈ {income, expense, transfer}

Check: amount > 0

Check: if type = 'transfer' then counterparty_account_id IS NOT NULL and category_id IS NULL

Check: if type ∈ {'income', 'expense'} then counterparty_account_id IS NULL

Check: counterparty_account_id ≠ account_id

Receipts(receipt_id, transaction_id, file_name, file_type, storage_location, uploaded_at)

PK: receipt_id

FK: transaction_id references Transactions(transaction_id)

Unique: transaction_id (this enforces the 1-to-1 relationship — each transaction has at most one receipt)

TransactionTags(transaction_id, tag_id)

PK: (transaction_id, tag_id) (composite primary key)

FK: transaction_id references Transactions(transaction_id)

FK: tag_id references Tags(tag_id)

nontrivial functional dependencies:

Users FDs:

$\text{user_id} \rightarrow \text{email, username, password_hash, full_name, created_at}$

$\text{email} \rightarrow \text{user_id, username, password_hash, full_name, created_at}$

$\text{username} \rightarrow \text{user_id, email, password_hash, full_name, created_at}$

(email and username each determine everything because they are unique — any unique attribute is a candidate key)

Accounts FDs:

$\text{account_id} \rightarrow \text{user_id, name, type, initial_balance, created_at}$

$(\text{user_id, name}) \rightarrow \text{account_id, type, initial_balance, created_at}$

(The composite (user_id, name) is a candidate key because they are unique — a user can't have two accounts with the same name.)

Categories FDs:

$\text{category_id} \rightarrow \text{user_id, name, type}$

$(\text{user_id, name}) \rightarrow \text{category_id, type}$

Tags FDs:

$\text{tag_id} \rightarrow \text{user_id, name}$

$(\text{user_id, name}) \rightarrow \text{tag_id}$

Transactions FDs:

$\text{transaction_id} \rightarrow \text{account_id, counterparty_account_id, category_id, type, amount, transaction_date, entry_date, description}$

(account_id and category_id alone doesn't determine anything because one account has many transactions)

Receipts FDs:

receipt_id $\rightarrow$ transaction_id, file_name, file_type, storage_location, uploaded_at

transaction_id $\rightarrow$ receipt_id, file_name, file_type, storage_location, uploaded_at

(transaction_id is a candidate key here because it is unique constraint enforcing the 1-to-1 relationship with Transactions so it can determine all other attributes too)

TransactionTags FDs:

(No non-trivial functional dependencies.)

(The only FD is (transaction_id, tag_id) $\rightarrow$ (transaction_id, tag_id), which is trivial, the relation has only these two attributes and they together form the primary key, there is nothing non-trivial)

BCNF

Every relation is already in BCNF. for every non-trivial FD listed above, the left-hand side is a superkey of its relation.

Possible Refinements to the Schema

certain attribute names, constraints, and relation details may be adjusted after creating and testing realistic sample data. Building some example records is the next step to check if everything makes sense and to see if the constraints are correct or not or if all the FDs make sense or not.